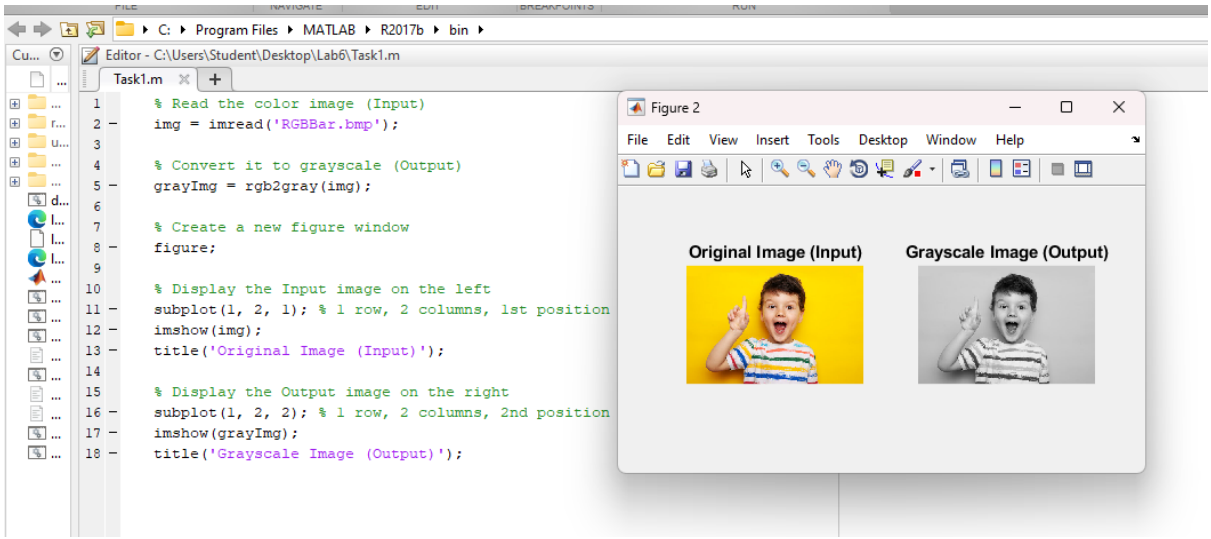
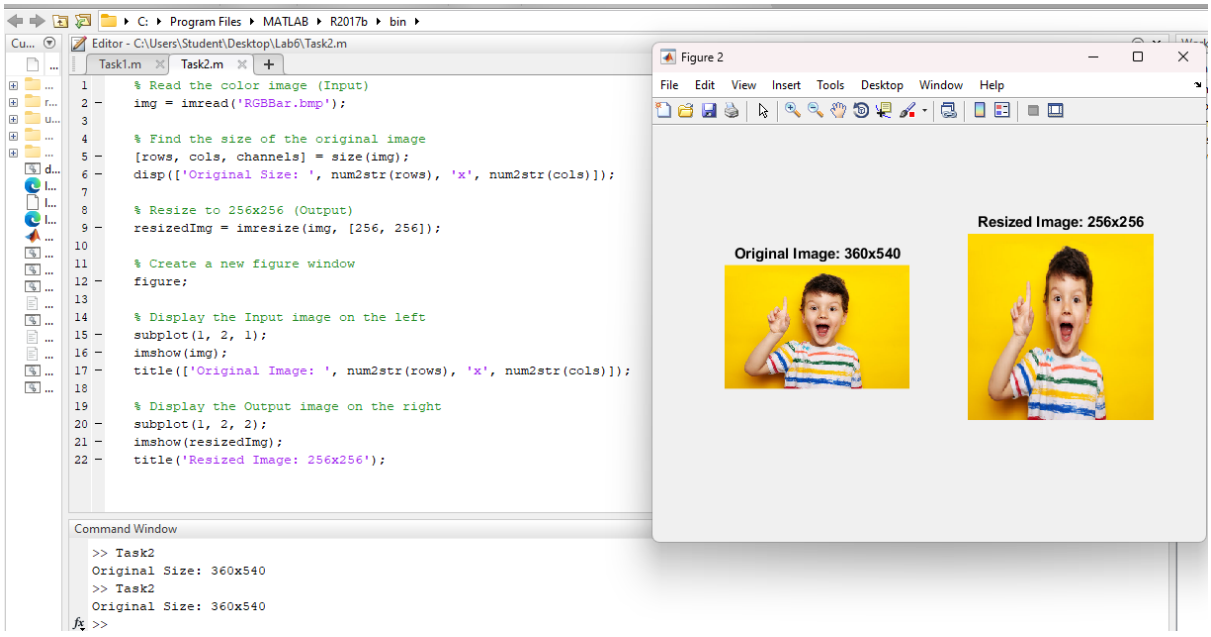
Roll No: CS23110 G3

DSP LAB 6:

TASK 1



TASK 2



TASK 3

Editor - C:\Users\Student\Desktop\Lab6\TASK3.m

Task1.m | Task2.m | TASK3.m | task4.m

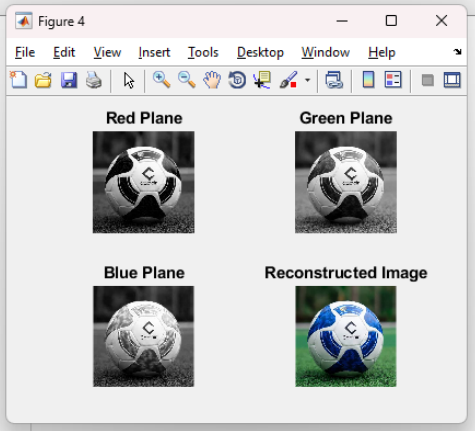
```
% Read a color image
img = imread('football2.jpg');

% Separate the RGB planes
R = img(:,:,1);
G = img(:,:,2);
B = img(:,:,3);

% Display each plane individually
figure;
subplot(2,2,1); imshow(R); title('Red Plane');
subplot(2,2,2); imshow(G); title('Green Plane');
subplot(2,2,3); imshow(B); title('Blue Plane');

% Recombine the planes
reconstructedImg = cat(3, R, G, B);

% Display the reconstructed image
subplot(2,2,4); imshow(reconstructedImg); title('Reconstructed Image');
```



TASK4

```
% 1. Read the original color images
origA = imread('image1.jpg');
origB = imread('image2.jpg');

% Resize Original B to exactly match Original A
[rows, cols, channels] = size(origA);
origB = imresize(origB, [rows, cols]);

% 2. Convert to grayscale, then to Black & White (binary)
bwA = im2bw(rgb2gray(origA));
bwB = im2bw(rgb2gray(origB));

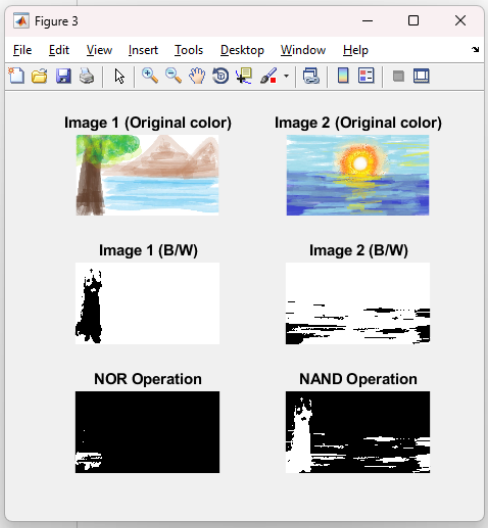
% 3. Perform logical NOR and NAND on the B/W images
norImg = ~(bwA | bwB);
nandImg = ~(bwA & bwB);

% 4. Display all 6 images in a 3x2 grid
figure;

% Top row: True Originals
subplot(3,2,1); imshow(origA); title('Image 1 (Original color)');
subplot(3,2,2); imshow(origB); title('Image 2 (Original color)');

% Middle row: Black and White inputs
subplot(3,2,3); imshow(bwA); title('Image 1 (B/W)');
subplot(3,2,4); imshow(bwB); title('Image 2 (B/W)');

% Bottom row: Logical Outputs
subplot(3,2,5); imshow(norImg); title('NOR Operation');
subplot(3,2,6); imshow(nandImg); title('NAND Operation');
```



TASK 5

```
% --- Part 1: Region of Interest (Masking) ---
% Make sure your file names match exactly what you saved
img = imread('football1.jpg');
mask = imread('mask.bmp');

% Safety Check 1: Convert main image to grayscale ONLY if it is RGB
if size(img, 3) == 3
    grayImg = rgb2gray(img);
else
    grayImg = img; % It's already grayscale!
end

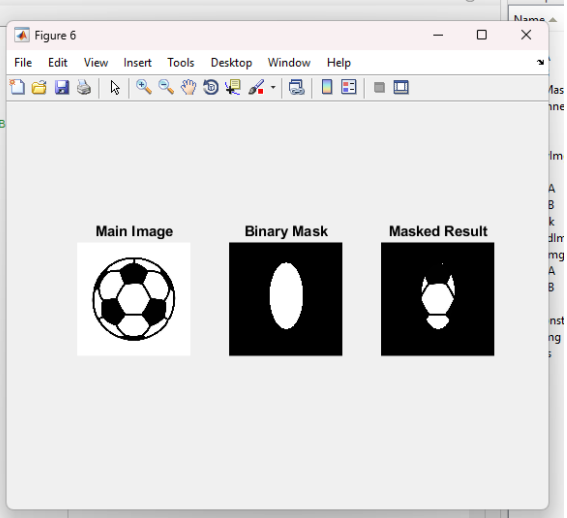
% Safety Check 2: Convert mask to grayscale ONLY if it is RGB
if size(mask, 3) == 3
    mask = rgb2gray(mask);
end

% Resize mask to perfectly match the main image dimensions
[rows, cols] = size(grayImg);
mask = imresize(mask, [rows, cols]);

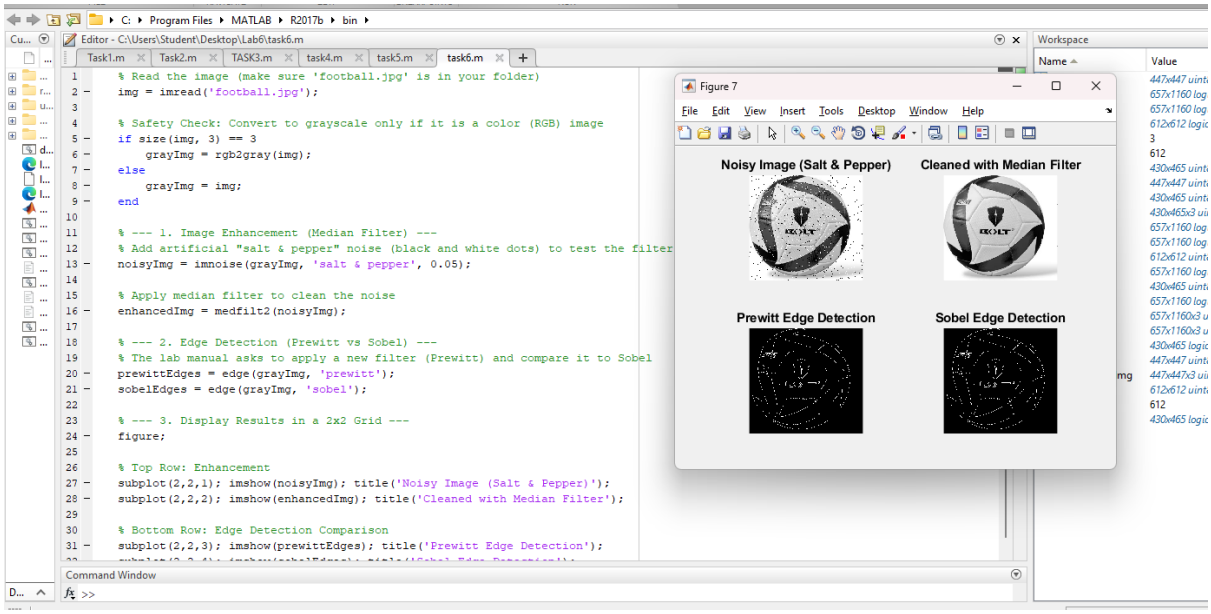
% Convert mask strictly to binary (1s and 0s)
bwMask = im2bw(mask);

% Multiply original by the binary mask
roiImg = uint8(grayImg) .* uint8(bwMask);

% Display Part 1 Results
figure;
subplot(1,3,1); imshow(grayImg); title('Main Image');
subplot(1,3,2); imshow(bwMask); title('Binary Mask');
subplot(1,3,3); imshow(roiImg); title('Masked Result');
```



TASK 6


Program Files ▸ MATLAB ▸ R2017b ▸ bin ▸
Editor - C:\Users\Student\Desktop\Lab6\task6.m
Task1.m
Task2.m
TASK3.m
task4.m
task5.m
task6.m
% Read the image (make sure 'football.jpg' is in your folder)
img = imread('football.jpg');
% Safety Check: Convert to grayscale only if it is a color (RGB) image
if size(img, 3) == 3
grayImg = rgb2gray(img);
else
grayImg = img;
end
% --- 1. Image Enhancement (Median Filter) ---
% Add artificial "salt & pepper" noise (black and white dots) to test the filter
noisyImg = imnoise(grayImg, 'salt & pepper', 0.05);
% Apply median filter to clean the noise
enhancedImg = medfilt2(noisyImg);
% --- 2. Edge Detection (Prewitt vs Sobel) ---
% The lab manual asks to apply a new filter (Prewitt) and compare it to Sobel
prewittEdges = edge(grayImg, 'prewitt');
sobelEdges = edge(grayImg, 'sobel');
% --- 3. Display Results in a 2x2 Grid ---
figure;
% Top Row: Enhancement
subplot(2,2,1); imshow(noisyImg); title('Noisy Image (Salt & Pepper)');
subplot(2,2,2); imshow(enhancedImg); title('Cleaned with Median Filter');
% Bottom Row: Edge Detection Comparison
subplot(2,2,3); imshow(prewittEdges); title('Prewitt Edge Detection');
Command Window
>>
Workspace
Name
Value
Figure 7
File Edit View Insert Tools Desktop Window Help
Noisy Image (Salt & Pepper)
Cleaned with Median Filter
Prewitt Edge Detection
Sobel Edge Detection